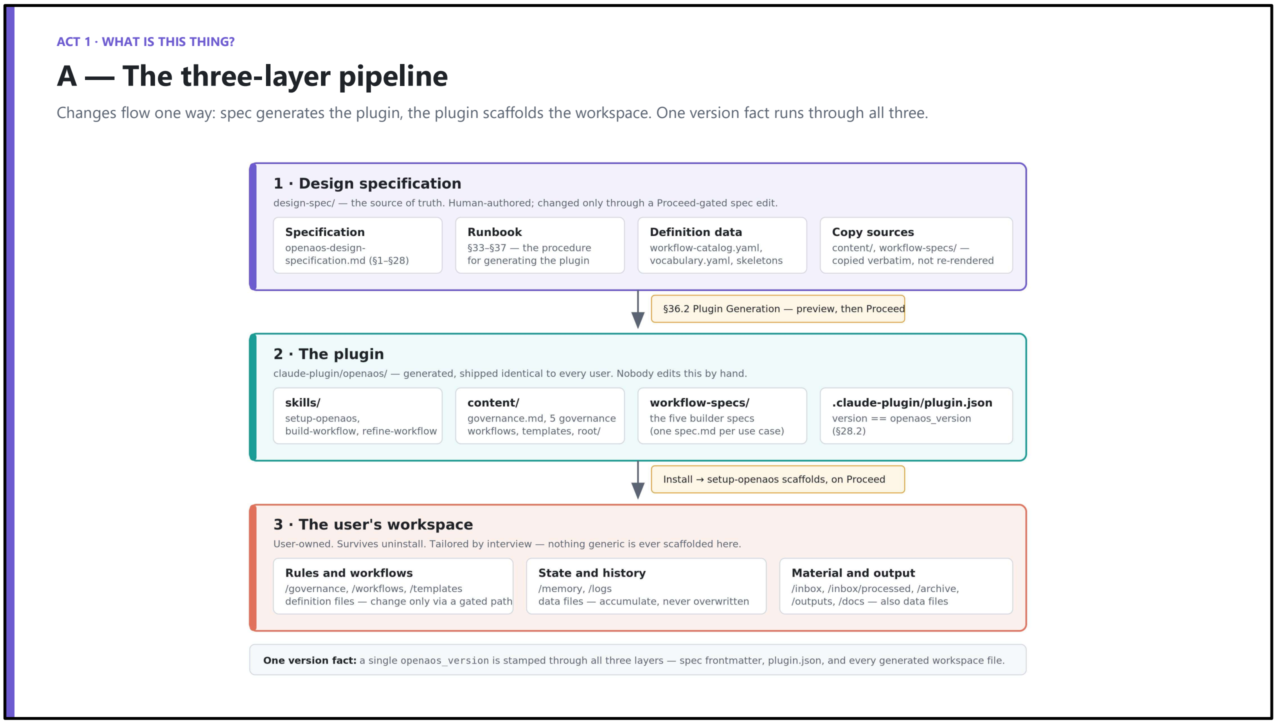
OpenAOS

Structure and processes

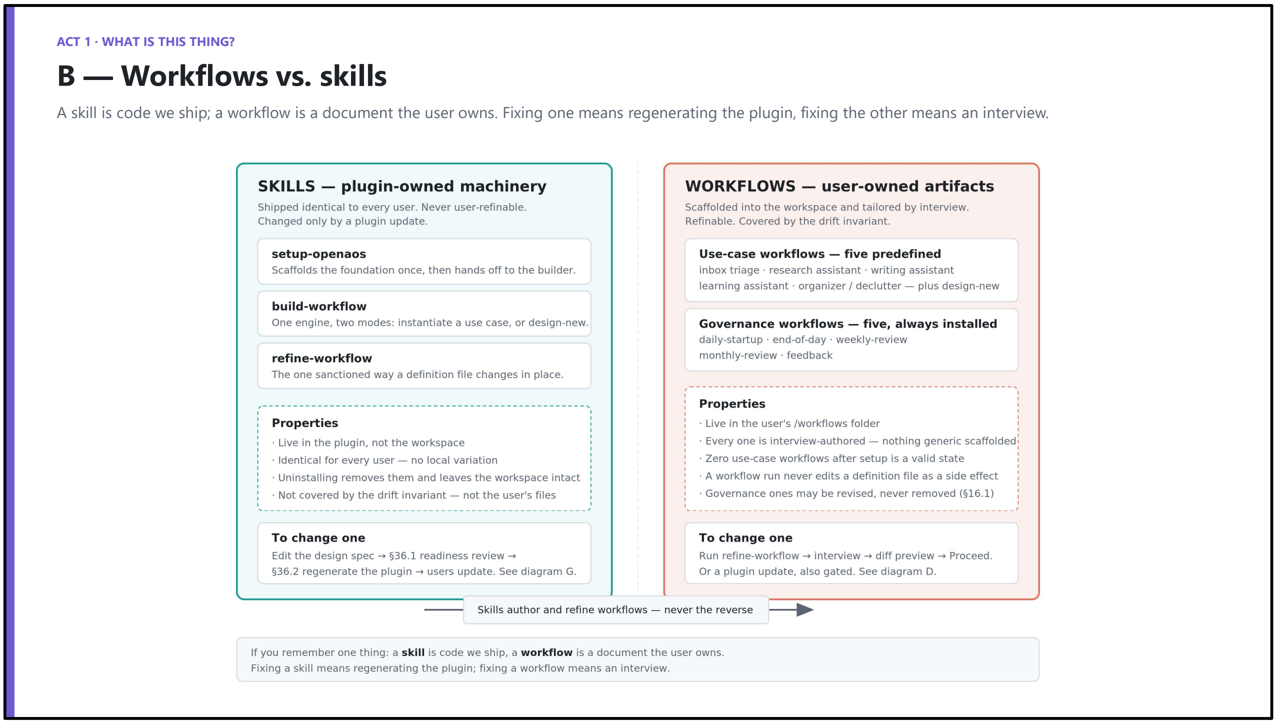
Act 1 · What is this thing? Act 2 · How does it behave? Act 3 · How do I change it safely?

Drawn from /design-spec as the source of truth · editable SVG sources in /diagrams/

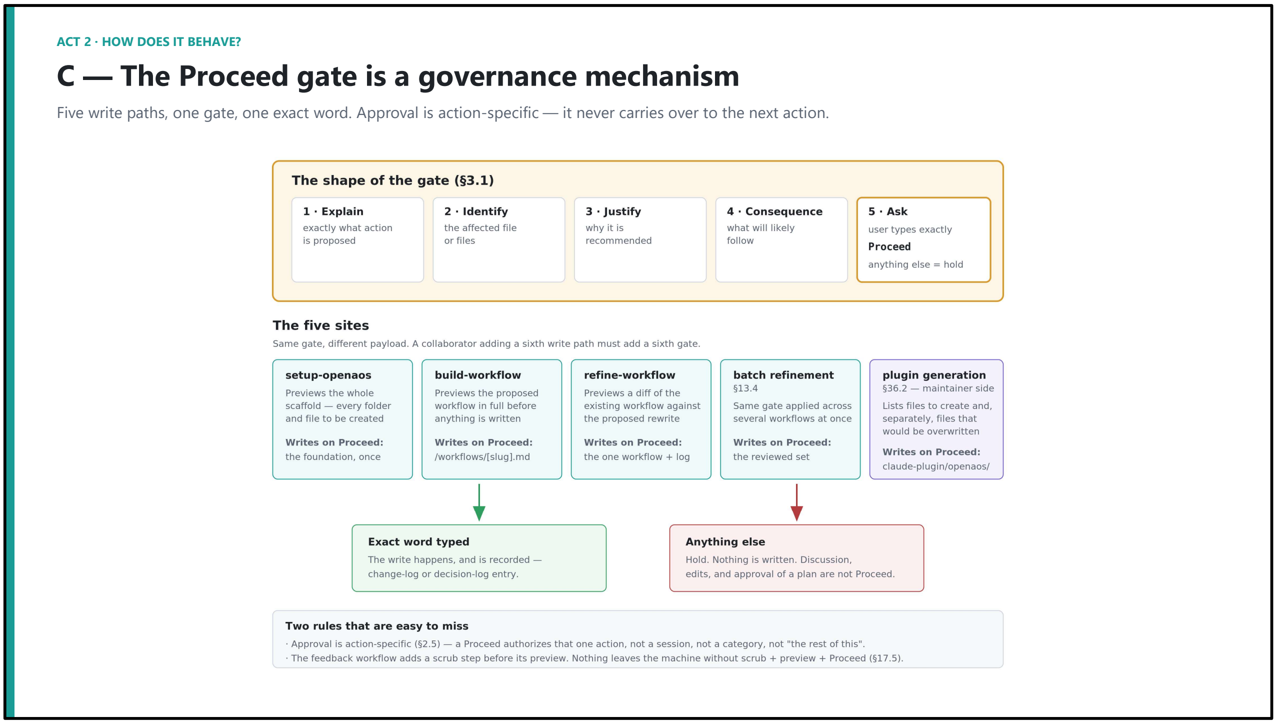
This deck introduces OpenAOS to someone who will be working on it but has not seen it before, and it is arranged as three questions rather than a tour of the file tree. Act 1 establishes what the project is: a design specification that generates a plugin, which in turn scaffolds a workspace the user owns. Act 2 covers how the resulting system behaves day to day — where it stops and asks permission, how a single workflow evolves, and what runs on a schedule. Act 3 covers the part that matters most to a contributor: how to change any of it without breaking the guarantees the earlier slides describe. Every diagram was drawn from design-spec/ as the source of truth, and the editable SVG sources sit alongside the deck in /diagrams/ — the images here are rendered PNGs, so amend the SVG and regenerate rather than editing inside PowerPoint.



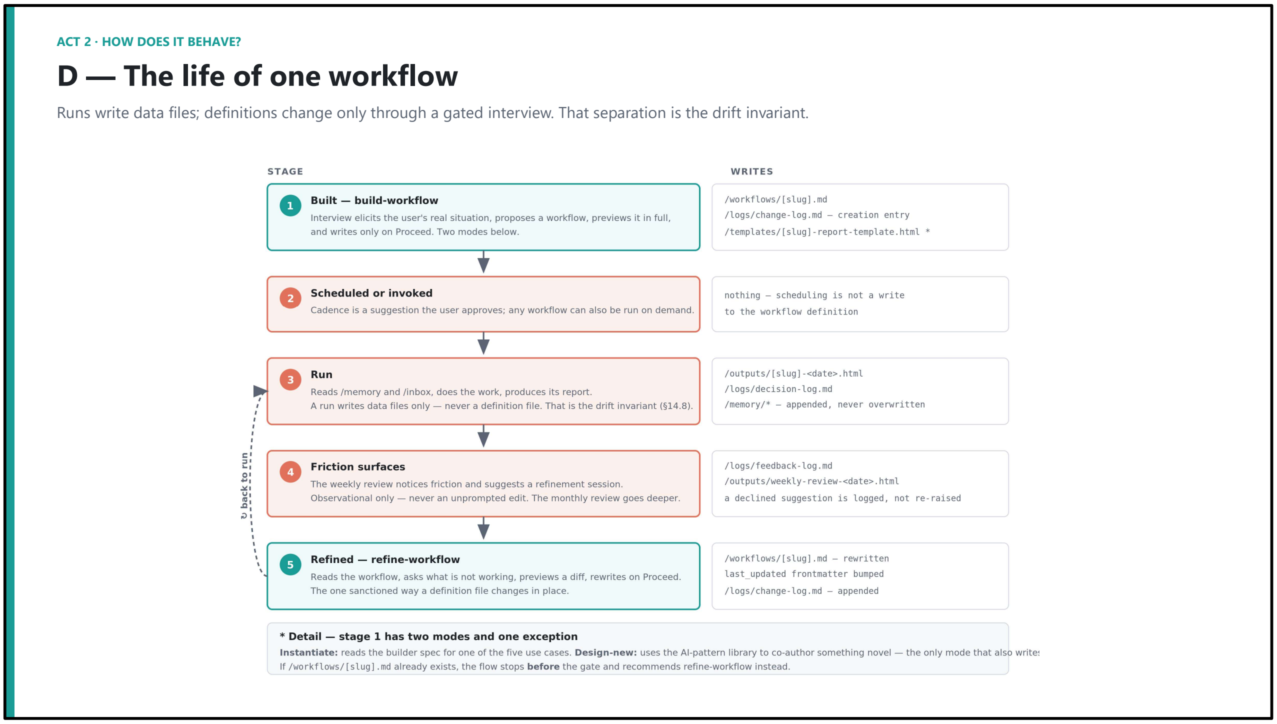
Three layers, and changes only ever flow downward. The design specification is human-authored and is the single source of truth; the plugin is generated from it and is never hand-edited; the user's workspace is scaffolded by the plugin at install time and thereafter belongs entirely to the user. Two things are worth pausing on. First, the content/ and workflow-specs/ folders are *copied* into the plugin verbatim rather than re-rendered, which is why generation can verify them with a byte-for-byte diff. Second, a single openaos_version is stamped through all three layers — spec frontmatter, plugin.json, and every generated workspace file — so there is exactly one version fact to reason about rather than a compatibility matrix. The practical consequence for a contributor is that any change to behaviour starts at the top of this diagram, never in the middle.



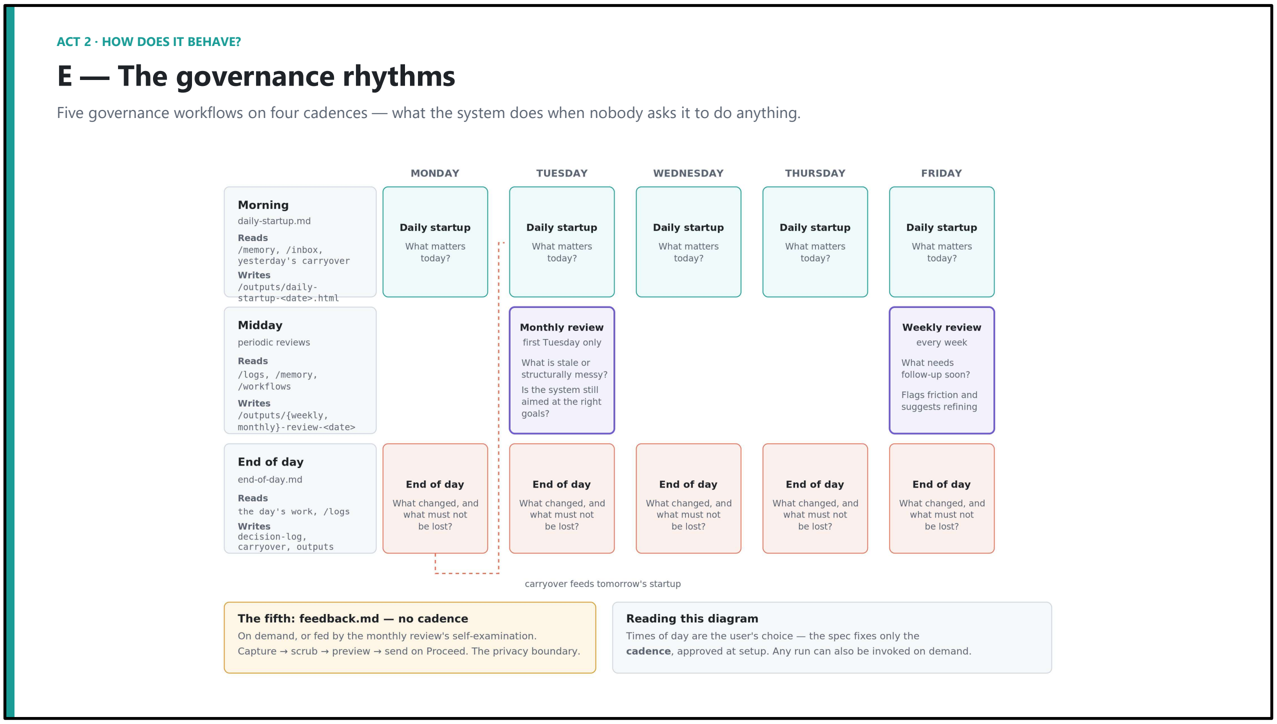
This is the distinction that governs every other conversation about the project, and confusing the two is the most common way to misunderstand a design decision. The plug-in contains three skills: /setup-openaos, /build-workflow, and /refine-workflow. These skills are plugin-owned machinery. They ship identical to every user, they are never user-refinable, and they change only through a plugin update. Workflows are user-owned documents that live in the user's /workflows folder, are authored through an interview so that no two users get the same file. Workflows are refinable at any time. Note the asymmetry at the bottom: skills author and refine workflows, never the reverse. The operational shorthand is that a skill is code we ship and a workflow is a document the user owns, which means fixing a skill requires regenerating the plugin while fixing a workflow requires nothing more than an interview using the /refine-workflow skill.



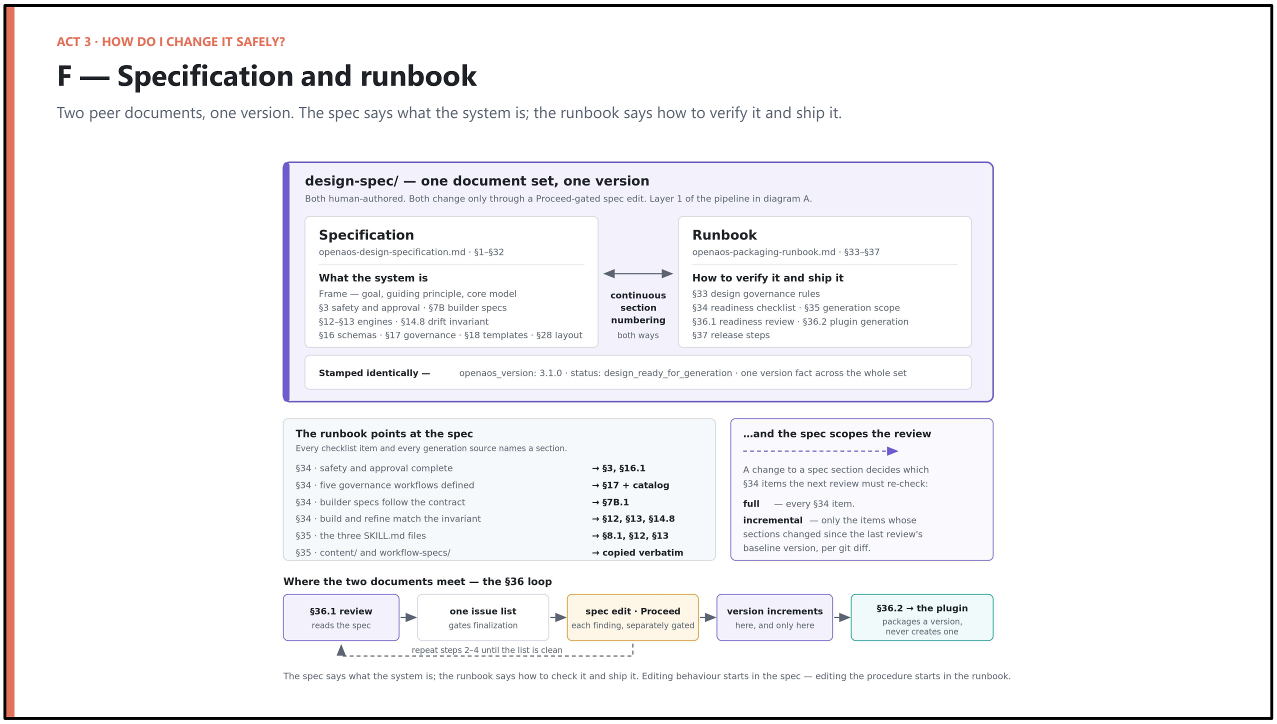
The system writes something irreversible in exactly five places, and all five stop at the same gate rather than each inventing its own confirmation style. The gate has a fixed five-part shape drawn from design-spec section §3.1: explain the action, identify the affected files, justify it, state the likely consequence, and ask the user to type the exact word. Anything other than that exact word is a hold — discussion, edits, and even enthusiastic approval of a plan are not Proceed. Two rules are easy to miss and worth stating aloud. Approval is action-specific under §2.5, so a Proceed authorizes one action and does not carry forward to a session or a category. And the feedback workflow inserts a scrub step before its preview, because nothing leaves the user's machine without scrub, preview, and Proceed — that sequence is an important privacy boundary.



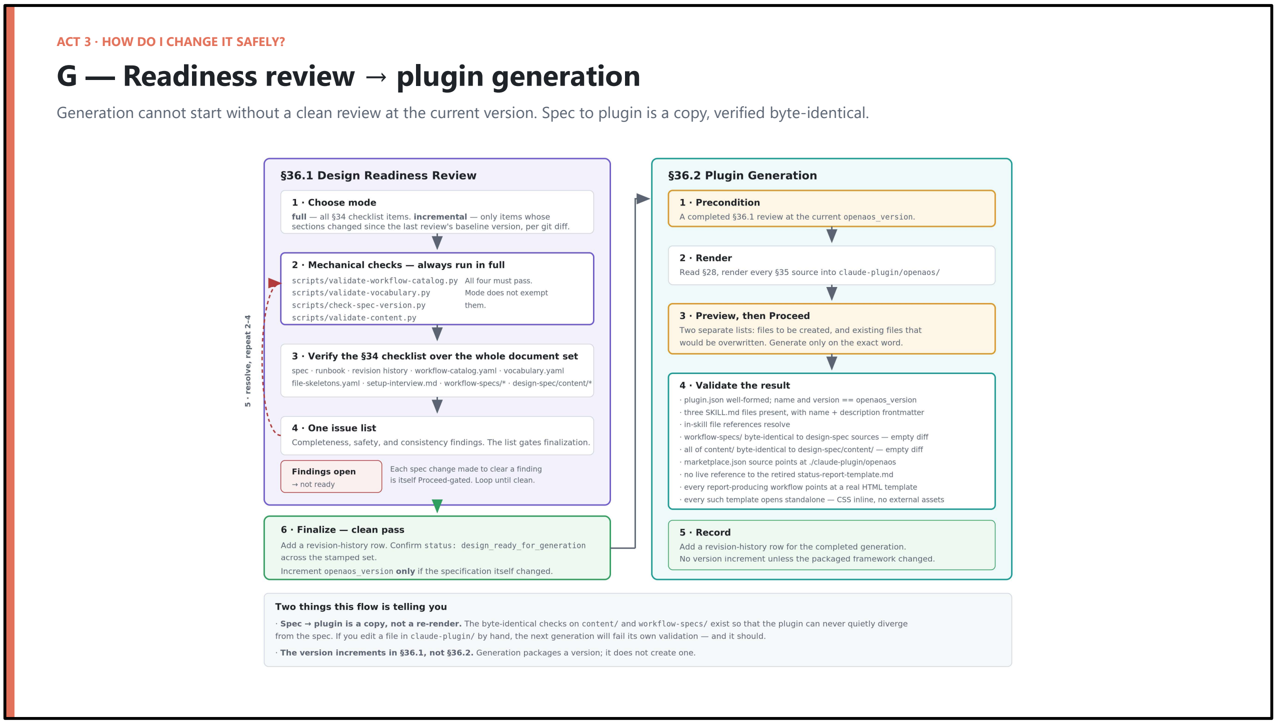
A workflow is not written once and left alone. It is expected to change over time to improve its behavior and adapt to changing requirements. This diagram shows the sanctioned path for that change alongside the artifacts each stage produces. It is built through an interview, either scheduled or invoked, run repeatedly, observed for friction by the weekly and monthly reviews, and eventually refined. This is an endless learning loop. Notice that runs write only data files while the workflow definition itself is untouched. That separation is the drift invariant from design-spec §14.8, and it is what keeps updates and refinements clean reviewed changes rather than three-way merges. Two details at the bottom are easy to trip over: 1) only design-new mode also writes a report template, and 2) if the target workflow file already exists the builder stops before the gate and recommends refinement instead.



This is what the system does automatically on a pre-defined schedule. Five governance workflows run on four cadences: a daily startup that asks what matters today, an end-of-day pass that captures what changed and what must not be lost, a weekly review that surfaces follow-ups and flags workflows producing friction, and a monthly review that asks the deeper question of whether the system is still aimed at the right goals. The dashed line traces the carryover from one day's close into the next day's startup, which is the mechanism that keeps obligations from evaporating overnight. The specification fixes only the *cadence*, not the clock times — those are proposed at setup and approved by the user. Scheduled workflows can also be invoked on demand. The fifth workflow, feedback, has no cadence at all and is triggered on demand or by the monthly review's self-examination.



The design spec and the runbook are peers, not layers. They are both are human-authored, neither is generated from the other, and their section numbering is deliberately continuous (spec §1–§32, runbook §33–§37) so cross-references resolve in both directions. The division is that the specification says what the system *is* and the runbook says how to verify it and ship it, which means a change in behavior starts in the spec and a change in procedure starts in the runbook. The middle band is worth reading aloud: every §34 checklist item and every §35 generation source names a specific spec section, so the runbook never restates the design, it only points at it. The traffic runs the other way too, because which spec sections changed is exactly what decides whether the next review is full or incremental. The bottom strip is the loop where the two documents actually meet, and it contains the detail people get wrong most often: findings from the review become Proceed-gated edits *to the spec*, and the version increments during the review, not during generation. Generation packages a version; it never creates one. One maintenance note when presenting: this diagram hard-codes openaos_version: 3.1.0 as an example only.



This is the maintainer's flow, and it is drawn as one diagram rather than two because the second half cannot start without the first: Runbook §36.2 step 1 is a precondition requiring a completed §36.1 review at the current version. The review runs four mechanical validators — which execute in full regardless of whether the review itself is full or incremental — then verifies the §34 checklist across the whole document set, collects everything onto a single-issue list, and loops until that list is clean, with each remedial spec change itself Proceed-gated. Generation then renders, previews, and validates. The validation list is worth reading closely, because two of its items are structural rather than cosmetic: content/ and workflow-specs/ must be byte-identical to their design-spec sources. That is a deliberate tripwire — hand-editing anything under claude-plugin/ will fail the next generation, and it should. Note also that the version increments during the review, not during generation; generation packages a version, it does not create one.

ACT 3 · HOW DO I CHANGE IT SAFELY?

H — Who may write what

Setup writes almost everything, once. Build and refine write one workflow each. Runs write only data.

PATH	KIND	setup-openaos	build-workflow	refine-workflow	a workflow run
/governance/governance.md	**definition**	●	○	◐	○
/workflows/[governance].md ×5	**definition**	●	○	◐	○
/workflows/[use-case].md	**definition**	○	●	●	○
/templates/*.html *.md	**definition**	●	◐	○	○
/docs/user-guide.html	**projection**	●	○	○	●
/memory/*	**data**	◐	○	○	●
/logs/change-log.md	**data**	◐	●	●	○
/logs/{decision,feedback}-log.md	**data**	◐	○	○	●
/outputs/[slug]-<date>.html	**data**	○	○	○	●
/inbox/**, /archive/**	**data**	◐	○	○	●
claude-plugin/openaos/**	**plugin**	no skill and no workflow may write here — only §36.2, on the maintainer's machine			

Legend

- ● writes freely, within its own gate
- ◐ restricted — see the notes at right
- ○ never writes here

Setup writes almost everything, but exactly once.
Build writes one workflow. Refine writes one workflow. Runs write only data.

The restrictions, spelled out

· **setup on data files** — creates them once, empty or seeded. A re-run or plugin update never overwrites them.

· **refine on governance** — may tune cadence, inputs, output shape, emphasis. May not delete a governance workflow, remove a gate, bypass the feedback scrub-preview-Proceed sequence, or weaken it.

· **build on templates** — design-new mode also writes a report template.

· **setup on change-log** — seeds it; later changes append, never rewrite.

This slide is a reference table rather than a narrative, and it answers the question a new contributor asks when they are about to write code: am I allowed to touch this path? Read the KIND column first, because it determines the rules — definition files change only through a gated path, data files accumulate and are never overwritten by an update, and the plugin tree is off-limits to everything except the maintainer-side generation flow. The shape that emerges is a clean division of labor between the plugin skills: /setup-aos writes almost everything but exactly once, /build-workflow and /refine-workflow each write a single workflow file plus a log entry, and workflow runs write only data. The amber panel spells out the four restricted cases, of which the most consequential is refinement's boundary on governance — it may tune cadence, inputs, output shape, and emphasis, but it may not delete a governance workflow, remove an approval gate, bypass the feedback scrub sequence, or weaken the governance config.